

Task Management Platform - Detailed Design & Interview Guide

1. Project Overview

Goal

Build a production-style backend for task management where users can register, login, create tasks, assign tasks, track status, search, filter and paginate results.

Tech Stack

Python, FastAPI, PostgreSQL, SQLAlchemy, Redis, JWT, Docker, Docker Compose.

2. High Level Architecture

Flow

Client → FastAPI Router → Service Layer → Repository Layer → PostgreSQL.
Frequently accessed responses are cached in Redis.
JWT middleware authenticates every protected request.

Why this architecture?

Separating routers, services and repositories keeps business logic independent from HTTP and database code.

3. Folder Structure

Recommended

```
app/  
main.py  
core/  
config.py  
security.py  
db/  
database.py  
models/  
schemas/  
repositories/  
services/  
api/  
middleware/  
utils/
```

4. Database Design

Users Table

id, name, email(unique), password_hash, role, created_at

Tasks Table

id, title, description, status, priority, assigned_to(FK), created_by(FK), due_date, created_at, updated_at

Relationships

One user can create many tasks. One user can be assigned many tasks.

5. Authentication

Registration

Validate input, hash password using bcrypt, store user.

Login

Verify password, generate JWT access token, optionally refresh token.

Protected APIs

Bearer token validated before controller execution.

6. RBAC

Roles

Admin can manage every task. User can manage only own/assigned tasks.

7. API Endpoints

Endpoints

POST /register
POST /login
GET /tasks?page=1&limit;=10
POST /tasks
PUT /tasks/{id}
DELETE /tasks/{id}
PATCH /tasks/{id}/status
GET /tasks?status=TODO&priority;=HIGH&search;=backend

8. Pagination, Filtering & Search

Pagination

LIMIT/OFFSET or cursor-based. Prevents loading thousands of rows.

Filtering

status, priority, assignee, due date.

Search

ILIKE or full-text search over title/description.

9. Redis

Usage

Cache GET /tasks and GET /tasks/{id}.

Cache Invalidation

Delete cache whenever task is created, updated or deleted.

TTL

5–10 minutes for list endpoints.

10. SQLAlchemy

Why ORM?

Safer queries, relationship mapping, migrations, maintainability.

Layers

Router -> Service -> Repository -> SQLAlchemy Session

11. Docker

Dockerfile

Packages FastAPI app with dependencies.

docker-compose

Runs FastAPI + PostgreSQL + Redis together.

12. Validation & Errors

Validation

Pydantic models validate request payloads.

Errors

Global exception handlers return consistent JSON.

13. Logging & Security

Logging

Request logs, error logs, audit logs.

Security

Password hashing, JWT expiry, CORS, environment variables, SQL injection prevention through ORM.

14. Scalability

Horizontal Scaling

Run multiple FastAPI instances behind Nginx/load balancer.

Database

Read replicas, indexes, connection pooling.

Caching

Redis reduces DB load.

Future

Kafka for async notifications, Celery for background jobs, AWS deployment, Kubernetes orchestration.

15. Interview Questions

Common Questions

Why FastAPI over Flask?

Why PostgreSQL?

Why Redis?

Why JWT?

How is RBAC implemented?

How does SQLAlchemy work?

How would you scale to 1M users?

How would you introduce Kafka?

How would you deploy on AWS?

How would Kubernetes help?

16. What to Say in Interview

Project Explanation

I designed a layered FastAPI backend with JWT authentication, RBAC, PostgreSQL, SQLAlchemy ORM, Redis caching, Dockerized deployment, pagination, filtering, search, centralized exception handling and modular architecture. The design focuses on scalability, maintainability and production readiness.

Future Enhancements

Implement refresh tokens, email notifications, WebSockets for live updates, Kafka event publishing, background workers, audit trail, file attachments, OpenAPI documentation, CI/CD pipeline, AWS deployment (EC2, RDS, ElastiCache), Kubernetes, monitoring with Prometheus/Grafana and distributed tracing.